

Prototype 2 — Advanced Architectural Structure

Prototype 2 — Advanced Architectural Structure

1. Team
2. Software System
 - 2.1. Name
 - 2.2. Logo
 - 2.3. Description
3. Architectural Structures
 - 3.1. Component-and-Connector (C&C) Structure
 - 3.2. Deployment Structure
 - 3.3. Layered Structure
 - 3.4. Decomposition Structure
4. Prototype
 - 4.1. Source Repositories
 - 4.2. Prerequisites
 - 4.3. Environment Configuration
 - 4.4. Bring up the full stack
 - 4.5. Smoke test
 - 4.6. Open the app
 - 4.7. Public access (Cloudflare Pages + Tunnel)
 - 4.8. CI and branch protection
5. Non-Functional Requirement Traceability

Prototype 2 — Advanced Architectural Structure

Software Architecture — 2026-I Universidad Nacional de Colombia

1. Team

Team Name: 1D

Members:

- **Juan David Loaiza Reyes** — General Orchestrator (`chat-orch`, Rust)
- **Víctor Daniel Díaz Reyes** — Tenant Service (Go), Compliance (Python)

- **Nicolás Zuluaga Galindo** — Tenant Service (Go)
- **Manuel Eduardo Díaz Sabogal** — Tenant Service & Frontend (Go / TypeScript)
- **Daniel Libardo Díaz Gonzalez** — Conversation / Chat (Go) & Agent Runtime (TypeScript)
- **Julián Andrés Vaquiro Moreno** — Frontend (TypeScript), Email Service (Java)

(Hospital-MP, RabbitMQ broker, and Compliance are shared work between members.)

GitHub organization (source of truth for all repositories): <https://github.com/UNagent-1D>

Repo	Upstream URL
Umbrella (deployment)	https://github.com/UNagent-1D/dev-runner
chat-orch	https://github.com/UNagent-1D/chat-orch
Tenant	https://github.com/UNagent-1D/Tenant
conversation-chat	https://github.com/UNagent-1D/conversation-chat
agent-runtime	https://github.com/UNagent-1D/agent-runtime
Hospital-MP	https://github.com/UNagent-1D/Hospital-MP
FrontEnd	https://github.com/UNagent-1D/FrontEnd
UN_email_send_ms	https://github.com/UNagent-1D/UN_email_send_ms
UN_message_broker_mb	https://github.com/UNagent-1D/UN_message_broker_mb
Compliance	in-tree under dev-runner/Compliance/

2. Software System

2.1. Name

Un Agent — Asesores en Salud — multi-tenant conversational-AI administration platform, hospital-appointment vertical slice.

2.2. Logo



Un Agent
ASESORES EN SALUD

2.3. Description

Un Agent is a multi-tenant conversational-AI platform that lets different organisations (*tenants*) configure and deploy AI agents tailored to their business domain. The Prototype 2 vertical slice is specialised for a hospital scheduling domain: end-users (patients) interact with the agent through a server-rendered web chat console (Next.js SSR) or a Telegram bot to list doctors, check schedules, book appointments, and cancel them. Tenant administrators access an admin dashboard — shipped in the same frontend application — that exposes operational KPIs (conversation volume, resolution rate, CSAT) and tenant/user management tooling.

The system follows a **microservices** design organised around a single front-door **Agent Orchestrator** (`chat-orch`). The orchestrator terminates every inbound channel (web, Telegram), authenticates the caller against the **Tenant** service, drives the per-turn LLM workflow through the **Agent Runtime**, persists conversation state to MongoDB Atlas, and emits telemetry to the internal **Compliance** service (which replaces the Prototype-1 *Metrics* service while preserving its wire contract). Two new subsystems extend the platform:

- **UN_email_send_ms** — outbound transactional email through SendGrid, with every attempt persisted to a dedicated audit Mongo (`email_events`).

- **UN_message_broker_mb** — a RabbitMQ broker that materialises the asynchronous chat-request / chat-result path, decoupling the orchestrator's HTTP loop from background tool execution.

Domain: Customer-service automation — second prototype, still specialised to hospital appointment scheduling (Clínica San Ignacio mock), now with real Postgres-backed seeding instead of in-memory mocks.

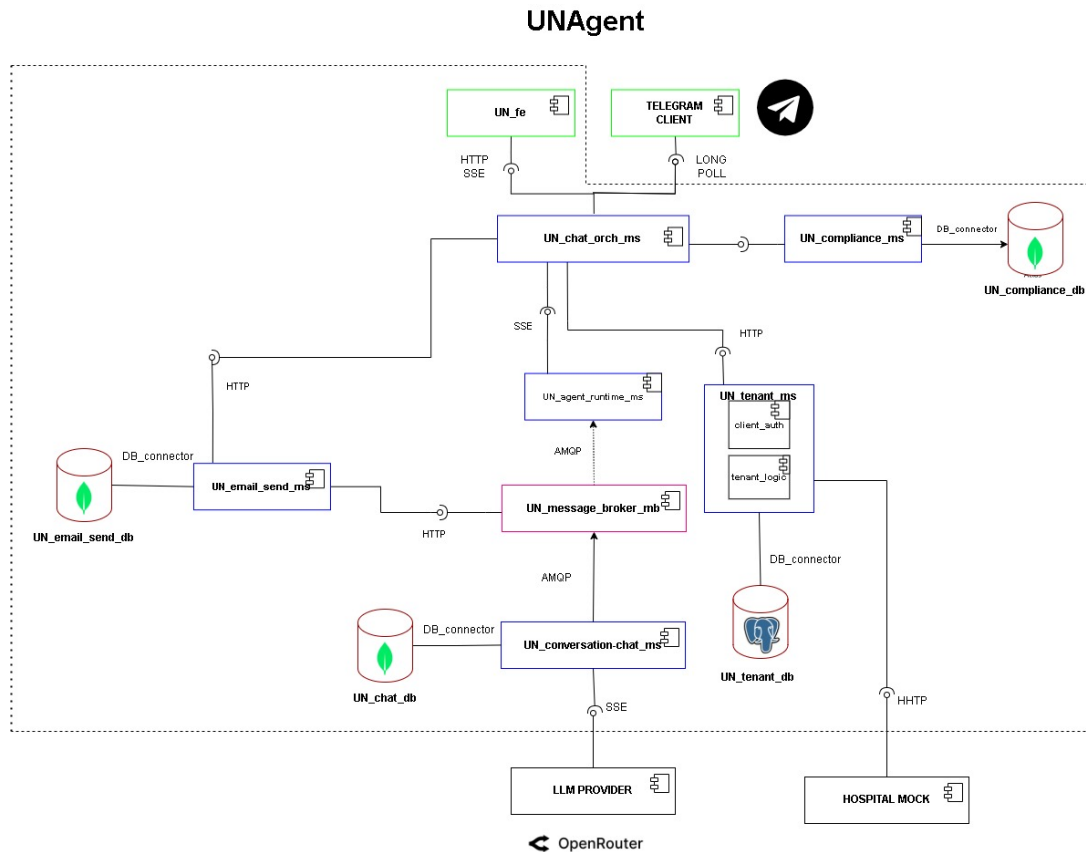
Prototype 2 functional scope:

- Web front-end built with **Next.js + Server-Side Rendering** to satisfy the "presentation must follow SSR" rule.
 - Hospital domain backed by a **PostgreSQL** instance (`hospital-postgres`) with idempotent migrations + seed data applied via `/docker-entrypoint-initdb.d`.
 - **Compliance** service (Python/FastAPI) exposing the legacy Metricas wire contract plus `/v1/event` and `/v1/feedback` audit endpoints persisted to MongoDB.
 - **Email** service (Java 21 / Spring Boot 3.3) for outbound transactional email via SendGrid, with every attempt persisted to a local audit Mongo (`email-mongo`).
 - **RabbitMQ** broker (`UN_message_broker_mb`) wired into both `agent-runtime` and `conversation-chat` to materialise the asynchronous turn pipeline (`chat_requests` and `chat_results` queues, declared via `definitions.json`).
 - Container-oriented deployment: every service ships a Dockerfile and the umbrella `dev-runner` orchestrates the full topology with `docker-compose`. The demo runs on a **local Linux host** fronted by **Cloudflare Pages** (FrontEnd) + **Cloudflare Tunnel** (backend ingress — outbound only, no inbound port forward).
-

3. Architectural Structures

3.1. Component-and-Connector (C&C) Structure

3.1.1. C&C View



Source: team draw.io export. Each `UN_*_ms` component below maps 1:1 to a deployment container in §3.2.

The dashed rectangle delimits the platform boundary. Five external systems sit outside it: **Telegram Bot API**, **MongoDB Atlas**, **Supabase** (managed Postgres), **OpenRouter** (LLM gateway), and **SendGrid** (email). Inside the boundary, the **Agent Orchestrator** (`UN_chat_orch_ms`) is the only component exposed to web clients; every other backend service is reachable only through it, through the Agent Runtime (`UN_agent_runtime_ms`), or (for analytics reads and admin login) through `chat-orch` and `Tenant` directly.

3.1.2. Architectural Styles and Patterns

The Prototype-2 architecture combines six complementary styles. Each solves a different concern.

1. Microservices (primary style). The system is decomposed into independently deployable services (`chat-orch`, `Tenant`, `conversation-chat`, `agent-runtime`, `Hospital-MP`, `Compliance`, `email-send`, `FrontEnd`), each with its own repository, technology stack, container image, and release cycle.

Services communicate exclusively over the network. This style directly satisfies the project's distributed-architecture and multi-language requirements.

2. Backend-For-Frontend (BFF) / API Gateway. The Agent Orchestrator (`chat-orch`) is the single HTTP front-door for every chat client (web console, Telegram). The frontend never talks directly to `agent-runtime`, `conversation-chat`, or `Hospital-MP`; it only knows about `chat-orch`, `Tenant`, and `Compliance`. This concentrates CORS scope, auth, and rate-limiting in one place and lets internal services evolve without coordinating releases with the UI.

3. Layered runtime — Agent Orchestrator → Agent Runtime → Conversation. The chat path is split into three logical layers, each owning a distinct concern:

- **Agent Orchestrator** (`chat-orch`, Rust) — transport, auth, fan-out to support services (`Tenant`, `Compliance`).
- **Agent Runtime** (`agent-runtime`, TypeScript) — per-turn LLM tool-calling loop, agent profile registry (ACR stub), and session adapter to `conversation-chat`. Owns the prompt and tool schemas.
- **Conversation/Chat** (`conversation-chat`, Go) — durable session state (MongoDB Atlas), history, and the actual streaming HTTP call to OpenRouter.

4. End-to-end Server-Sent Events (SSE) streaming. A single SSE chain carries assistant tokens from the model to the browser, with every hop preserving streaming semantics. Each hop is an HTTP `text/event-stream` body with a 15-second keep-alive ping. Avoids buffering the full LLM reply before showing it to the user.

5. Publish/Subscribe (asynchronous messaging) — *new in P2*. A RabbitMQ broker (`UN_message_broker_mb`) declares two durable queues — `chat_requests` and `chat_results` — that decouple the orchestrator's synchronous HTTP loop from the background tool execution path inside `conversation-chat::worker` and `agent-runtime::broker`. This satisfies the "asynchronous processes" non-functional requirement and gives the platform headroom for multiple concurrent tool runs without blocking the front door.

6. Persistence per service (Database-per-service). Each service owns its own data store (`Tenant` → Supabase, `Conversation` → Atlas, `Hospital` → `hospital-postgres`, `Compliance` + `Email` → `email-mongo`). No two services share a schema; cross-service queries always go through HTTP.

3.1.3. Architectural Elements

The system exposes **two presentation** components, **six logic** components, **one orchestration/messaging** component, and **four data** components, plus three external SaaS providers.

Presentation components

Component	Stack	Responsibility
FrontEnd (web)	Next.js 15 (SSR) + React 19 + TypeScript + Tailwind + shadcn/ui + Zustand + TanStack Query + recharts	Server-rendered chat console + tenant-admin dashboard. Hits <code>chat-orch</code> , <code>Tenant</code> , and <code>Compliance</code> only. KPI charts refetch every 10 s.
Telegram channel (in-process within <code>chat-orch</code>)	Rust + request long-poll	Inbound bot messages via <code>getUpdates</code> ; outbound via <code>sendMessage</code> . Same runtime as <code>/v1/chat</code> , so the LLM loop is reused.

Logic components

Component	Stack	Responsibility
<code>chat-orch</code> — Agent Orchestrator	Rust 2021, Axum 0.7, Tokio	BFF / gateway. Terminates REST + SSE from the frontend and Telegram long-poll. Drives the per-turn LLM workflow through <code>agent-runtime</code> . Emits fire-and-forget telemetry to <code>Compliance</code> .
<code>agent-runtime</code> — Agent Runtime	TypeScript, Express 5, Node.js 20, AMQP client (<code>amqplib</code>)	LLM tool-calling loop + ACR stub + tenant stub + session proxy + RabbitMQ producer/consumer.
<code>conversation-chat</code> — Conversation Service	Go + Gin + mongo-driver + AMQP (<code>amqp091-go</code>)	Durable session and per-turn history. Streams the OpenRouter response. Async worker drains <code>chat_requests</code> / produces <code>chat_results</code> .
<code>Tenant</code> — Tenant & RBAC Service	Go + Gin + bcrypt + JWT (HS256)	JWT login, tenant CRUD, user roster, RBAC. Source of truth for tenant identity.
<code>Compliance</code> — KPI + Audit Service	Python 3.12 + FastAPI + motor (async Mongo)	In-memory KPI counters and daily buckets per tenant. Back-compatible with the Metrics wire contract (<code>/conversation/chat</code> , <code>/feedback/csat</code> , <code>/stats/kpis</code> , <code>/stats/timeseries</code>) plus <code>/v1/event</code> and <code>/v1/feedback</code> audit endpoints persisted to Mongo.
<code>email-send</code> — Email Service	Java 21 + Spring Boot 3.3 + Spring Data MongoDB + sendgrid-java + jjwt	Outbound transactional email via SendGrid. Persists every attempt to <code>email-mongo</code> (<code>email_events</code> collection).

Communication / Orchestration component

Component	Stack	Responsibility
<code>UN_message_broker_mb</code> — Message Broker	RabbitMQ 3- management + <code>definitions.json</code> declarative bootstrap	Hosts the <code>chat_requests</code> and <code>chat_results</code> durable queues. Plays the publish/subscribe connector role; consumed by <code>agent- runtime</code> and <code>conversation- chat::worker</code> .

Data components (four total — relational + NoSQL covered with margin)

Component	Type	Hosting	Owner	Role
Supabase PostgreSQL	Relational	External SaaS	Tenant	<code>tenants</code> , <code>users</code> , <code>user_tenants</code> (RBAC join). Connected via the Session Pooler URI (IPv4- compatible).
MongoDB Atlas	NoSQL (document)	External SaaS	<code>conversation- chat</code>	Durable session records, full per-turn history, tool-call traces.
<code>hospital- postgres</code>	Relational	Local container, <code>hospital- postgres- data volume</code>	Hospital data layer	<code>doctors</code> and <code>appointments</code> tables. Schema + seed (the Hospital-MP migration scripts and a thin Flask query wrapper at <code>:8092</code>) applied on first boot via <code>/docker-entrypoint- initdb.d</code> .
<code>email- mongo</code>	NoSQL (document)	Local container, <code>email- mongo- data volume</code>	<code>email-send</code> , Compliance	Two databases on one Mongo instance: <code>email_audit.email_events</code> and <code>UN_compliance_db.audit_logs</code> .

External services (outside the platform boundary)

Component	Protocol	Role
OpenRouter	HTTPS + SSE	OpenAI-compatible LLM gateway. Default model <code>nvidia/nemotron-3-super-120b- a12b:free</code> .
Telegram Bot API	HTTPS long-poll	Inbound messaging channel (<code>getUpdates</code> + <code>sendMessage</code>).
SendGrid v3	HTTPS	Email delivery. Sandbox-mode active in dev.

3.1.4. Connectors and Relations

#	Connector	Protocol	Where
1	REST (sync request/response)	JSON / HTTP/1.1	FrontEnd → chat-orch / Tenant / Compliance; chat-orch → agent-runtime / Tenant / Compliance / Hospital-MP; agent-runtime → conversation-chat / Hospital-MP.
2	Server-Sent Events (SSE)	HTTP/1.1 text/event-stream	End-to-end chat-token streaming chain.
3	HTTP long-polling	HTTPS GET getUpdates? timeout=30	Telegram ↔ chat-orch.
4	AMQP 0.9.1 (Pub/Sub)	TCP, RabbitMQ wire protocol	agent-runtime ↔ chat_requests / chat_results ↔ conversation-chat::worker.
5	PostgreSQL wire protocol	TCP (Supabase Session Pooler / local)	Tenant ↔ Supabase; Hospital query wrapper ↔ hospital-postgres.
6	MongoDB wire protocol	TCP (Atlas SRV / local)	conversation-chat ↔ Atlas; email-send and Compliance ↔ email-mongo.

3.2. Deployment Structure

3.2.1. Deployment View

The deployment view is split into two complementary diagrams — both strictly UML 2.x compliant: rounded «device» nodes for hardware, «executionEnvironment» for runtimes, «component» for services, «artifact» (folded-corner) for deployable files, **solid lines** for communication paths and **dashed lines** for dependencies / «deploy» / «manifest» / «persists».

Diagram 1 of 2 — System Context *Devices, Cloudflare edge, and external SaaS. Color-coded edges: **blue** = HTTPS/HTTP, **purple** = Cloudflare Tunnel (mTLS), **green** = TCP/TLS to managed databases.*

UNAgent — Deployment (1/2): System Context
Devices, edge, and external SaaS.
Color coded for readability.

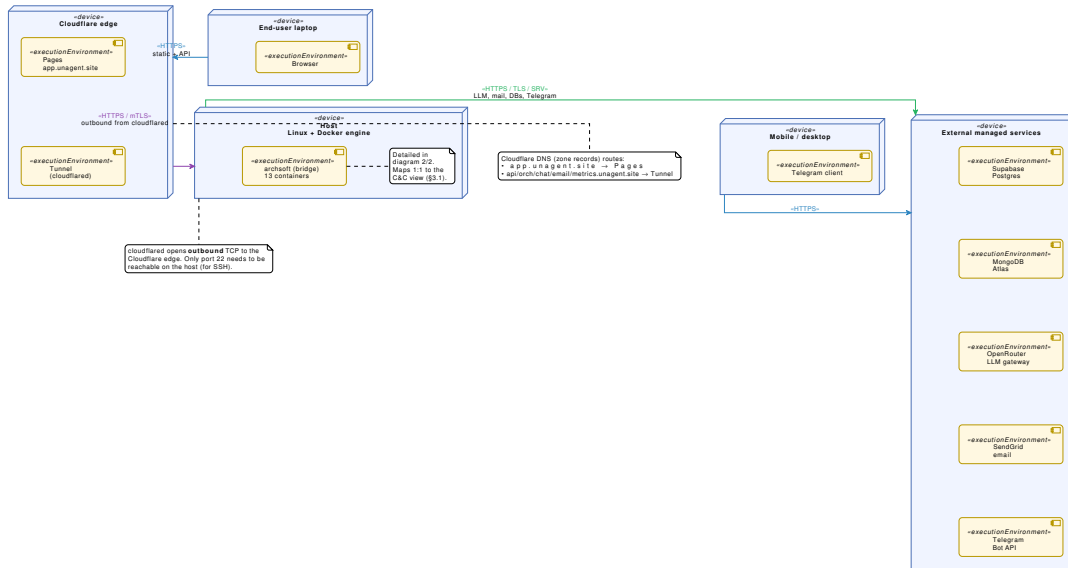
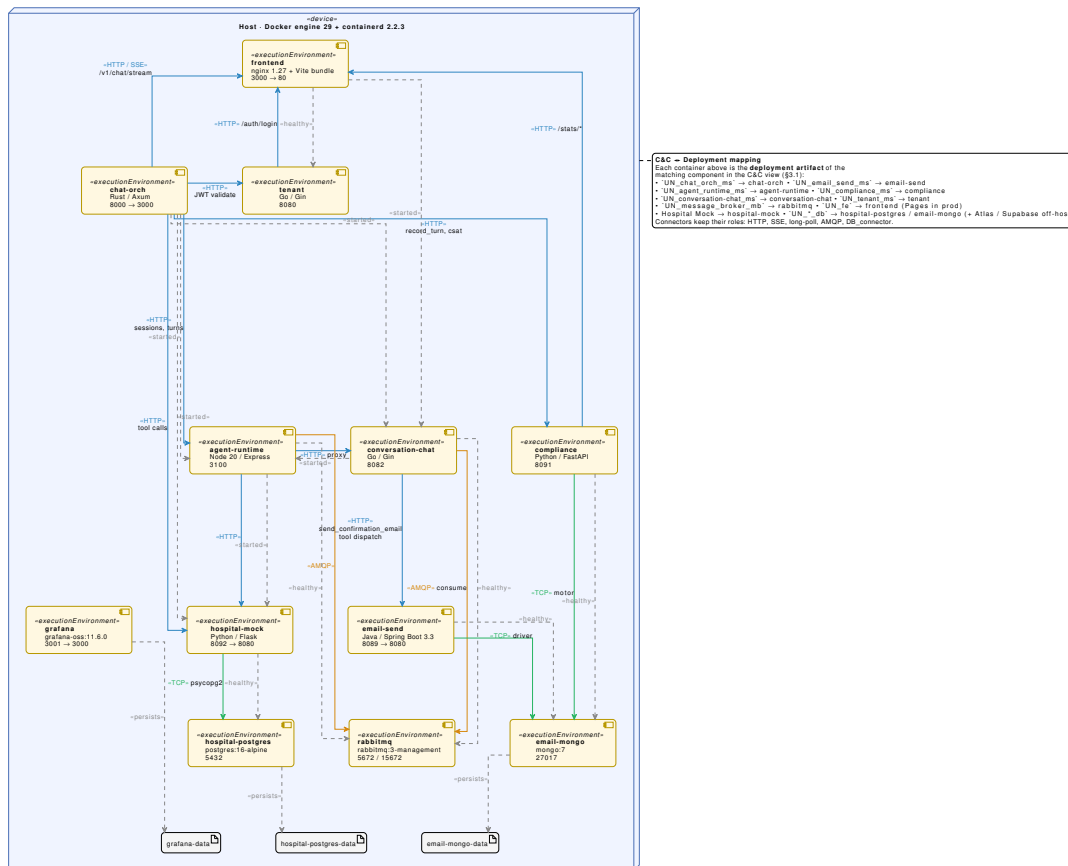


Diagram 2 of 2 — Host-internal Everything inside the *archsoft* Docker network. Color-coded edges: **blue** = HTTP, **green** = TCP (databases), **orange** = AMQP, **grey dashed** = depends_on / persists.

UNAgent — Deployment (2/2): Host-internal view
Everything inside the archsoft Docker network.
Color coded for readability — maps 1:1 to the C&C view (§3.1).



C&C ↔ Deployment cross-reference. Each container in the host-internal diagram is the deployment artifact of the matching component in the C&C view (§3.1):

C&C component	Deployment container
UN_chat_orch_ms	chat-orch
UN_agent_runtime_ms	agent-runtime
UN_conversation-chat_ms	conversation-chat
UN_tenant_ms	tenant
UN_compliance_ms	compliance
UN_email_send_ms	email-send
UN_message_broker_mb	rabbitmq
UN_fe	frontend (and Cloudflare Pages in prod)
Hospital Mock	hospital-mock
UN_compliance_db	email-mongo (DB UN_compliance_db) + Atlas
UN_email_send_db	email-mongo (DB email_audit)
UN_chat_db	MongoDB Atlas
UN_tenant_db	Supabase Postgres

Connectors retain their roles across views: **HTTP**, **SSE**, **long-poll**, **AMQP**, and **DB_connector** in the C&C view appear as the same protocol stereotypes on the deployment edges.

3.2.2. Architectural Elements and Relations

Devices. The deployment topology has four «device» nodes:

1. **End-user laptop** (Browser engine «executionEnvironment»).
2. **Mobile / desktop** running a Telegram client.
3. **Cloudflare edge** (managed) — hosts two execution environments:
 - **Cloudflare Pages** serves the FrontEnd's Next.js SSR build at `app.<domain>`.
 - **Cloudflare Tunnel** runs `cloudflared` as a systemd unit on the host and exposes the backend services via stable HTTPS sub-domains (`api`, `orch`, `chat`, `metrics`, `email`).
4. **Host** — a local Linux workstation running Docker engine 29 + containerd 2.2.3 + runc 1.4. The Docker engine is itself an «executionEnvironment» that hosts a bridge network `archsoft`, which in turn hosts every service container and its named volumes.

Containers

Each container is shown with its image, host:container port mapping, healthcheck, and the deployable «artifact» that runs inside it (binary, JAR, bundle, or interpreted source). Multi-stage builds are labelled with both the *builder* and *runtime* base images. See diagram 2 for the per-container details.

Communication paths.

- **Edge ingress** (solid lines, between devices):
 - Browser ↔ Cloudflare edge: «HTTPS» (static SSR pages + REST/SSE).

- Cloudflare Tunnel ↔ Host: «HTTPS / mTLS» (outbound from the host to the Cloudflare edge — no inbound port forward needed on the host).
- Telegram client ↔ Telegram Bot API: «HTTPS» (managed channel).
- Host ↔ Telegram Bot API / OpenRouter / SendGrid / Supabase / Atlas: «HTTPS» and «TCP/TLS».
- **Intra-host** (between components in the bridge network, all over Docker DNS):
 - REST / SSE on every chat-path link.
 - AMQP 0.9.1 between `agent-runtime`, `conversation-chat::worker`, and `rabbitmq`.
 - TCP for Postgres and Mongo client connections.
- **Lifecycle dependencies** (dashed): every `depends_on` declared in `docker-compose.yml`, with the explicit health condition («healthy» for services with a healthcheck, «started» otherwise).
- **Persistence** (dashed «persists»): two named volumes survive container recreation — `hospital-postgres-data` and `email-mongo-data`.

Deployment patterns applied.

1. Container-per-service. Each microservice ships a single Dockerfile producing one image; one container per service runs in the network. No two services share a process or filesystem.

2. Edge tunnel (Cloudflare Tunnel). Instead of opening inbound ports on the host (which would require a public IP, dynamic DNS, and TLS certificate management), the host runs an outbound-only `cloudflared` daemon that registers itself with the Cloudflare edge. Cloudflare's PoPs terminate TLS for the public hostnames and forward decrypted traffic over the tunnel. This is a managed implementation of the *edge-proxy* deployment pattern.

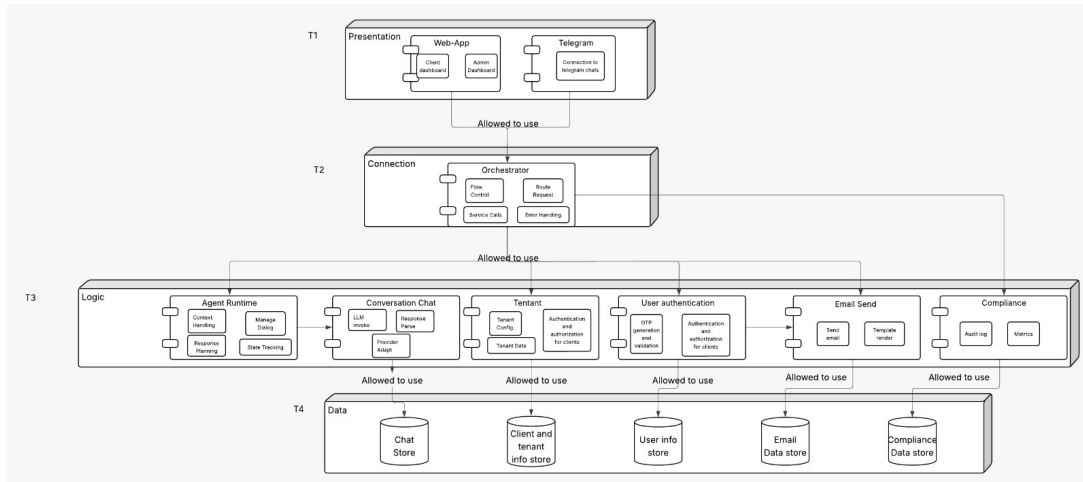
3. Database-per-service. Each service binds to its own dedicated data store. Cross-service data only flows through HTTP/gRPC calls, never via shared schemas. The `email-mongo` container exposes two logically separate databases (one for `email-send`, one for `Compliance`) but each microservice still owns its own database name

- collection family.

4. Init-container-style schema bootstrapping. The `hospital-postgres` container leverages PostgreSQL's `/docker-entrypoint-initdb.d` convention to apply `schema.sql` and `seed.sql` exactly once, on first boot of the volume. To re-seed, drop the named volume and recreate the container — the same pattern documented in the Postgres image.

3.3. Layered Structure

3.3.1. Layered View



3.3.2. Architectural Elements and Relations

The platform is organised in **four tiers** (T1 — T4). The «Allowed to use» arrows in the diagram flow strictly **downward**: each tier may only call the tier directly below it; no tier ever calls a tier above it, and no tier may skip a level.

Tier	Layer	Members
T1	Presentation	Web-App (Client Dashboard + Admin Dashboard) and Telegram (connection to Telegram chats)
T2	Connection	Orchestrator (Flow Control, Route Request, Service Calls, Error Handling)
T3	Logic	Agent Runtime · Conversation Chat · Tenant · User Authentication · Email Send · Compliance
T4	Data	Chat Store · Client and Tenant Info Store · User Info Store · Email Data Store · Compliance Data Store

Internal modules per T3 component.

- **Agent Runtime** — Context Handling, Manage Dialog, Response Planning, State Tracking.
- **Conversation Chat** — LLM Invoke, Response Parse, Provider Adapt.
- **Tenant** — Tenant Config, Tenant Data.
- **User Authentication** — OTP Generation and Validation, Authentication and Authorization for Clients.
- **Email Send** — Send Email, Template Render.
- **Compliance** — Audit Log, Metrics.

Allowed-to-use relations.

- **T1** → **T2** — Web-App and Telegram can only invoke the Orchestrator in T2; they never reach the logic or data tiers directly.

- **T2 → T3** — the Orchestrator dispatches to all six T3 logic components, applying flow control, routing and service-call error handling around each fan-out.
- **T3 → T4** — every T3 component owns exactly one T4 data store:
 - Conversation Chat → Chat Store
 - Tenant → Client and Tenant Info Store
 - User Authentication → User Info Store
 - Email Send → Email Data Store
 - Compliance → Compliance Data Store

No upward call exists, and no tier may skip its neighbour (T1 → T3 is **not** allowed in this view; everything funnels through T2).

3.3.3. Architectural Patterns Used

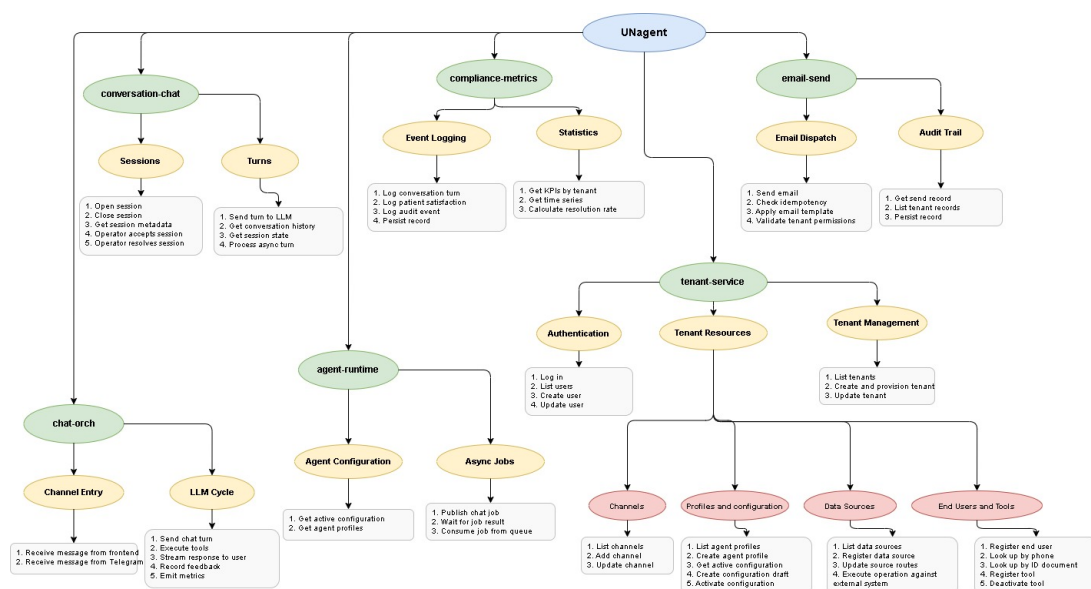
1. Strict layered architecture. Each tier may only call the tier directly below it. The «Allowed to use» arrows enforce a single descent path (T1 → T2 → T3 → T4); no upward calls and no skipped levels.

****2. Orchestration layer.** ** The Orchestrator is the sole T2 component, so every T1 client (web or Telegram) must traverse it before reaching application logic. This concentrates flow control, request routing, and error handling in one place.

3. Database-per-component. Each T3 logic component binds to exactly one T4 data store; no two components share the same store. Data isolation is enforced both at the architectural-rule level and through separate database/collection ownership in the running stack.

3.4. Decomposition Structure

3.4.1. Decomposition View



3.4.2. Architectural Elements and Relations

The platform decomposes into **eleven subsystems**, each with its own internal modules. Subsystems own their data stores; modules inside a subsystem may collaborate freely; cross-subsystem collaboration always goes through the subsystem's public façade (its HTTP API or its AMQP queue).

- 1. Presentation Subsystem** (`FrontEnd Next.js` + Telegram channel) — modules: `app/` SSR pages, Zustand auth store, axios + TanStack Query client, Telegram long-poll handler.
 - 2. Orchestration Subsystem** (`chat-orch`) — modules: HTTP router (`routes.rs`, Axum), LLM turn loop (`runtime.rs`), tool dispatcher (`hospital.rs`), SSE hub (`sse.rs`), in-memory session store (`session.rs`).
 - 3. Tenant & Identity Subsystem** (`Tenant`) — modules: auth handler (JWT HS256), RBAC middleware, tenant admin API (Gin), Postgres migrations.
 - 4. Agent Runtime Subsystem** (`agent-runtime`) — modules: ACR stub (`agents/hospital.ts`), tenant stub (`routes/tenant-stub.ts`), session proxy (`routes/proxy.ts`), AMQP broker client (`broker/connection.ts`).
 - 5. Conversation Subsystem** (`conversation-chat`) — modules: session service (`internal/service`), AMQP worker (`internal/worker`), Mongo adapter.
 - 6. Compliance Subsystem** (`Compliance`) — modules: legacy Metrics API (`back-compat`), compliance audit API (`/v1/event`, `/v1/feedback`), in-memory KPI store, Mongo audit writer.
 - 7. Hospital Data Subsystem** (`Hospital-MP`) — schema (`schema.sql`)
 - seed (`seed.sql`) for the `hospital-postgres` instance, plus a thin Flask query wrapper (`app.py`, `db.py` over `psycopg2`) that exposes the seeded catalogue to the agent's tool loop.
 - 8. Email Subsystem** (`UN_email_send_ms`) — modules: EmailController (REST), SendGrid provider (with retry + sandbox toggle), audit repository (Spring Data Mongo), JWT/stub auth filter.
 - 9. Async Messaging Subsystem** (`UN_message_broker_mb`) — modules: RabbitMQ broker, `chat_requests` queue, `chat_results` queue, `definitions.json` declarative bootstrap.
 - 10. Data Subsystem** — four owned databases (Supabase, Atlas, `hospital-postgres`, `email-mongo`). Each is bound to exactly one application-tier subsystem.
 - 11. Edge & Deployment Subsystem** (`dev-runner`) — `docker-compose.yml`, Cloudflare Pages project, Cloudflare Tunnel config, GitHub Actions CI workflow (`compose-validate` + Compliance tests ≥85%), and the branch-protection ruleset.
-

4. Prototype

4.1. Source Repositories

The platform is shipped as a **dev-runner** umbrella repository that pins each service repo as a git submodule. One clone with `--recurse-submodules` reproduces the exact working stack.

All repositories live under the UNagent-1D GitHub organization: <https://github.com/UNagent-1D>

Path in umbrella	Upstream repo	Language	Role
chat-orch/	UNagent-1D/chat-orch	Rust	Agent Orchestrator + SSE hub + Telegram long-poll
agent-runtime/	UNagent-1D/agent-runtime	TypeScript	Agent Runtime (ACR stub, tenant stub, session proxy, AMQP)
Tenant/	UNagent-1D/Tenant	Go	Auth + tenant admin API
conversation-chat/	UNagent-1D/conversation-chat	Go	Session + history service (Atlas + OpenRouter + AMQP worker)
Hospital-MP/	UNagent-1D/Hospital-MP	Python	Hospital schema + seed scripts and a Flask query wrapper for hospital-postgres
Compliance/	(in-tree)	Python	KPI service + audit log (replaces Metrics)
FrontEnd/	UNagent-1D/FrontEnd	TypeScript	Next.js 15 SSR admin dashboard + chat console

Path in umbrella	Upstream repo	Language	Role
UN_email_send_ms/	UNagent-1D/ UN_email_send_ms	Java	Outbound transactional email + audit
UN_message_broker_mb/	UNagent-1D/ UN_message_broker_mb	(RabbitMQ image)	Message broker with declared queues

```
# 1. Clone with submodules in one shot
git clone --recurse-submodules \
  git@github.com:UNagent-1D/dev-runner.git
cd dev-runner
```

To bump every submodule to its tracked upstream `main` later:

```
git submodule update --remote --merge
git add . && git commit -m "chore: bump submodules" && git push
```

4.2. Prerequisites

- **Docker** ≥ 24 and **Docker Compose v2**
- An **OpenRouter API key** (<https://openrouter.ai/keys>)
- A **MongoDB Atlas** cluster + SRV connection string
- A **Supabase** project — use the **Session Pooler** URI (IPv4); the direct URL is IPv6-only on the free tier
- A **SendGrid API key** (sandbox-mode is fine for the demo)
- (Optional) A **Telegram bot token** from @BotFather

4.3. Environment Configuration

```
cp .env.example .env
# Fill in:
#   OPENROUTER_API_KEY      = sk-or-v1-...
#   JWT_SECRET              = (long random string)
#   DATABASE_URL            = postgres://
#                           postgres:PWD@<pooler>.supabase.co
#                           :5432/postgres?sslmode=require
#   MONGO_URI               = mongodb+srv://
#                           USER:PWD@CLUSTER.mongodb.net/
#                           ?retryWrites=true&w=majority
#   MONGO_DB                = conversatory
#   SENDGRID_API_KEY        = SG.xxx (or any string with sandbox
#                           mode on)
#   EMAIL_FROM_DEFAULT      = noreply@unagent.local
#   TELEGRAM_BOT_TOKEN      (optional)
#   TELEGRAM_DEFAULT_TENANT_ID = demo-tenant
#
# Optional overrides (all have sane defaults in compose):
#   MONGO_URI_COMPLIANCE    = mongodb://email-mongo:27017
#   MONGO_DB_COMPLIANCE     = UN_compliance_db
#   HOSPITAL_DB_NAME        = hospital
```

```
# HOSPITAL_DB_USER = hospital
# HOSPITAL_DB_PASSWORD = hospital
```

4.4. Bring up the full stack

```
docker compose up -d --build
docker compose ps
```

All twelve containers should reach a healthy state within ~3 minutes (longest poles: chat-orch's Rust compile on first build, and email-send's Spring Boot warm-up).

4.5. Smoke test

```
curl -s http://localhost:8080/health # tenant
curl -s http://localhost:8000/health # chat-orch
curl -s http://localhost:8082/health # conversation-chat
curl -s http://localhost:3100/health # agent-runtime
curl -s http://localhost:8091/health # compliance
curl -s http://localhost:8089/health # email-send
curl -s http://localhost:8092/doctors # hospital-mock -
    returns 5 seeded doctors
curl -s http://localhost:3000/ # FrontEnd
```

4.6. Open the app

- **FrontEnd (web SSR):** <http://localhost:3000>
- **RabbitMQ management UI:** <http://localhost:15672> (default credentials per UN_message_broker_mb/definitions.json)

4.7. Public access (Cloudflare Pages + Tunnel)

The demo is reachable from any internet-connected device:

URL	Service
https://app.unagent.site	FrontEnd (Cloudflare Pages, edge-cached SSR build)
https://api.unagent.site	Tenant (login + admin)
https://orch.unagent.site	chat-orch (REST + SSE)
https://chat.unagent.site	conversation-chat (sessions + history)
https://metrics.unagent.site	Compliance (KPIs + audit)
https://email.unagent.site	email-send

The backend hostnames resolve to Cloudflare's edge and reach the local Docker stack via a `cloudflared` tunnel running as a `systemd` unit. No inbound port is opened on the host; the tunnel makes only outbound TCP connections to the Cloudflare edge.

4.8. CI and branch protection

- Every PR runs the umbrella CI workflow:
 - `docker compose config` validation (no submodule pulls required).
 - Compliance tests with `pytest --cov-fail-under=85` (currently passing at 98% line coverage on `Compliance/main.py`).
- The `main` branch is protected by a Cloudflare ruleset requiring one approving review, both CI checks green, no force-push, no deletion, linear history, and conversation resolution.

5. Non-Functional Requirement Traceability

Requirement	How the system meets it
Distributed architecture	Eight microservices + a broker, all communicating over the network in the <code>archsoft</code> Docker bridge network.
≥ 2 presentation components (one web)	<code>FrontEnd</code> (web, Next.js SSR) + Telegram channel (long-poll inside <code>chat-orch</code>).
Web frontend SSR	Next.js 15 with the App Router; pages rendered on the server before hitting the browser.
≥ 5 logic components	<code>chat-orch</code> , <code>tenant</code> , <code>agent-runtime</code> , <code>conversation-chat</code> , <code>Compliance</code> , <code>email-send</code> , plus the Hospital query wrapper — 6 services + 1 data wrapper .
≥ 1 communication / orchestration component	<code>chat-orch</code> (synchronous orchestration) + <code>RabbitMQ</code> (asynchronous Pub/Sub broker).
≥ 4 data components (relational + NoSQL)	Supabase Postgres + MongoDB Atlas + <code>hospital-postgres</code> + <code>email-mongo</code> — 4 total (2 relational, 2 NoSQL document).
≥ 1 component for asynchronous processes	<code>UN_message_broker_mb</code> <code>RabbitMQ</code> broker with <code>chat_requests</code> and <code>chat_results</code> queues, consumed by <code>agent-runtime</code> and <code>conversation-chat::worker</code> .
HTTP-based connectors	REST, SSE, HTTP long-polling — three distinct HTTP-family connectors plus AMQP for async.
≥ 5 general-purpose languages	Rust (<code>chat-orch</code>) · Go (<code>Tenant</code> , <code>conversation-chat</code>) · Python (<code>Hospital-MP</code> , <code>Compliance</code>) · TypeScript (<code>FrontEnd</code> , <code>agent-runtime</code>) · Java (<code>email-send</code>) — 5 languages .

Requirement	How the system meets it
Container-oriented deployment	Every service ships a <code>Dockerfile</code> ; the full stack is orchestrated via Docker Compose from <code>dev-runner</code> , running on a local Linux host fronted by Cloudflare Pages + Cloudflare Tunnel.